

87.42% accuracy on the Pathfinder task, surpassing the 71.40% accuracy of purely neural methods (Figure 3d).

2.3 Scalability and Programmability Challenges

The neurosymbolic solution poses a significant scalability challenge. While the neural component can utilize modern hardware accelerators like GPUs and TPUs, the symbolic component runs on CPUs alone. This presents a performance bottleneck, as in order to calculate derivatives, the symbolic engine must consider all possible predicted graphs and their associated probabilities. As the size of the input image or difficulty of line curvature increases, the number of possible structures and the size of their associated weights also grows exponentially, leading to a combinatorial explosion in the number of required computations.

An expert could write custom GPU kernels to accelerate this specific task, but this requires specialized knowledge of CUDA performance tuning. Instead, we seek to build a general, GPU-accelerated framework that accelerates any logic program used as part of a neurosymbolic pipeline, in order to make GPU acceleration widely accessible.

2.4 Our Results

Figure 3e shows the speedup of Lobster over the CPU baseline, Scallop, on the Pathfinder task. Scallop spends significant time on symbolic computation, while Lobster can perform the requisite combinatorial graph processing much quicker. Importantly, users do not need to change existing neurosymbolic programs to benefit from the acceleration. Lobster’s efficiency gains are enabled by mapping a significant fragment of Datalog to the GPU programming paradigm, and making judicious decisions for representing relations, parallelizing relational operators, and scheduling computation which we describe next.

How to Represent Relations? Lobster uses a flat, column-oriented layout in order to make optimal use of the GPU memory hierarchy. While performant CPU Datalog engines such as Soufflé make use of multi-layer data structures such as B-trees [21], our simple layout makes more sense in the context of GPU acceleration as column-oriented layouts are cache-friendly, and GPU programs are often memory-bound rather than compute-bound. Concretely, this means some of the simpler operations in the Pathfinder task, such as unioning the relation edge with the relation path to initially populate path, can reach close to 100% utilization of the GPU memory bus. This choice also suits the context of executing Datalog programs. For instance, the transitive closure operation is compiled to a series of query operations consisting of relational operations like *join* and *project* which operate on specific columns. As a result, columnar data allows more natural algorithm implementations. We discuss the memory layout further in Section 5.

How to Parallelize Relational Operators? Beyond memory layout, efficient algorithms are necessary to improve the performance of symbolic computations. In Lobster, the key insight is that Datalog programs are compiled to a core set of relational queries, and each of these queries can be individually parallelized to improve their performance on potentially massive inputs. For example, the rule on line 6 of Figure 3c is compiled to a query that includes joining the entire set of currently discovered paths against the base edge set. The size of the input to this join grows exponentially as the program iterates, so executing the join with data-parallelism with respect to its input is critical. We describe how our compiler exposes this parallelism in Section 3.

How to Handle Provenance Tags? Computing derivatives of the symbolic computation is necessary for training the neural network via gradient descent. This necessitates tracking the provenance of each fact in the Datalog program which involves computing over large and potentially complicated semiring tags. To adapt powerful but complex semirings like *diff-top-k-proofs* [26]—which tracks up to k proofs of arbitrary size for each fact—to the GPU, we leverage two insights. First, we observe that the max proof size can be statically determined at compile time. Second, we find that the *diff-top-1-proofs* semiring that tracks just one proof is sufficient for many programs; Lobster could also easily be extended to track larger k as well.

3 Language and Compiler

Lobster focuses on accelerating the Datalog back-end with GPU hardware. This poses a challenge, as the process of supporting rich reasoning is at odds with ensuring high performance. To achieve both of these goals, Lobster introduces a new intermediate language, APM (Abstract Parallel Machine), which is designed to simplify the process of compiling and optimizing Datalog programs for GPU execution. We assume an existing Datalog compiler is capable of converting a user-level program to a mid-level program based on relational algebra. From there, Lobster compiles the relational algebra down to an APM program that can be executed on the GPU. In this section, we describe the low-level sequential language APM and present the compilation process from the mid-level relational algebra language to APM.

3.1 Background

Relational Algebra Machine. We start by describing our compiler’s source language, the Relational Algebra Machine (RAM), which is based on the familiar language of *Relational Algebra* for expressing database queries [2]. The abstract syntax of RAM is shown in Figure 4. At a high level, executing a RAM program ϕ means sequentially executing each stratum $\phi_1, \dots, \phi_n$. Within each stratum, rules are iteratively applied to the extensional database (EDB), which contains the input facts, until a fix-point is reached. The newly derived

Table 1. A summary of instructions in the APM language. Lowercase latin characters represent registers, while capital latin characters represent scalar integers. We write $\overline{r}_n$ to denote a sequence of n registers and r_t to denote a register containing semiring tags. $\alpha_{n,m}$ denotes a function from n -tuples to m -tuples, σ a function $\tau \times \tau \rightarrow \tau$ for some type τ , and ρ a relation in the database.

Signature	Description
$\text{alloc}(\tau_1, \dots, \tau_n)(\overline{r}_n, S)$	Allocate n registers each of size S and types $\tau_1, \dots, \tau_n$. In practice, the type can be inferred based on usage and is elided.
$\overline{d}_m \leftarrow \text{eval}(\alpha_{n,m})(\overline{s}_n)$	Evaluate α on each row of $\overline{s}_n$.
$\overline{d}_n \leftarrow \text{gather}(i, \overline{s}_n)$	Gather rows of $\overline{s}_n$ based on indices i .
$d \leftarrow \text{gather}(\alpha_{n,1})(\overline{i}_n, \overline{s}_n)$	Gather rows of $\overline{s}_n$ based on indices $\overline{i}_n$ and reduce the resulting tuple with α .
$\text{store}(\rho)(\overline{s}_n, s_t)$	Store registers $\overline{s}_n$ and s_t as the columns and tags for relation ρ with arity n in the database.
$[\overline{s}_n, s_t] = \text{load}(\rho)()$	Loads the columns and tags of relation ρ with arity n from the database into registers $\overline{s}_n$ and s_t .
$d \leftarrow \text{build}(\overline{s}_n)$	Builds a hash index for the table with columns $\overline{s}_n$.
$\overline{d}_n \leftarrow \text{count}(\overline{b}_n, h, \overline{a}_n)$	Count the number of occurrences of each tuple in the table with columns $\overline{b}_n$ in the table with columns $\overline{a}_n$ via the hash index h .
$\overline{d}_n \leftarrow \text{scan}(s)$	Computes the exclusive prefix sum of register s .
$[d_l, d_r] \leftarrow \text{join}(W)(\overline{b}_m, \overline{a}_n, h, c, o)$	Produces the resulting indices from a W column join of two tables with columns $\overline{b}_m$ and $\overline{a}_n$ via the hash index h , histogram c , and histogram prefix sum o .
$\overline{d}_n \leftarrow \text{copy}(\overline{s}_n)$	Copies from register $\overline{s}_n$, truncating if the destination is smaller than the source.
$\overline{d}_n \leftarrow \text{sort}(\overline{s}_n)$	Lexicographically sorts the table with columns $\overline{s}_n$.
$[\overline{d}_n, s] \leftarrow \text{unique}(\sigma)(\overline{s}_n)$	Merges adjacent duplicate rows via σ from the table with columns $\overline{s}_n$, returning the number of unique elements s .
$\overline{d}_n \leftarrow \text{merge}(\overline{a}_n, \overline{b}_n)$	Merges two lexicographically sorted tables with columns $\overline{a}_n$ and $\overline{b}_n$.

(Predicate)	ρ
(Projection Fn.)	α
(Selection Fn.)	β
(Expression)	$\epsilon ::= \rho \mid \pi_\alpha(\epsilon) \mid \sigma_\beta(\epsilon) \mid \epsilon_1 \bowtie_n \epsilon_2$ $\mid \epsilon_1 \cup \epsilon_2 \mid \epsilon_1 \times \epsilon_2 \mid \epsilon_1 \cap \epsilon_2$
(Rule)	$\psi ::= \rho \leftarrow \epsilon$
(Stratum)	$\phi ::= \{ \psi_1, \dots, \psi_n \}$
(Program)	$\overline{\phi} ::= \phi_1; \dots; \phi_n$

Figure 4. The RAM language.

facts form the intensional database (IDB), which accumulates intermediate results produced by the program. Each rule $\rho \leftarrow \epsilon$ consists of a target relation ρ and a query ϵ . This query is a dataflow graph with many sources but only one sink. The operators in the graph are a core fragment of relational algebra operators, comprising project (π), select (σ), and join ($\bowtie$) as well as three set operators, union ($\cup$), product ($\times$), and intersect ($\cap$). Note that π and σ allow taking arbitrary projection or selection functions, while the join operation $\bowtie$ accepts the number of columns to perform join on. For this section, we focus on accelerating a single recursive stratum.

Provenance Semirings. Relational algebra programs can incorporate differentiable or probabilistic reasoning by tagging each fact with additional information such as probabilities or boolean formulas, as shown in prior work [26,

28]. More generally, *provenance semirings* [17] enable programmable semantics that allow tags from an arbitrary semiring. Formally speaking, a provenance semiring T is a 5-tuple $(T, \mathbf{0}, \mathbf{1}, \oplus, \otimes)$ where T is the space of tags (Figure 5a). $\oplus$ and $\otimes$ dictate how tags are combined through disjunction and conjunction operations. In Figure 5b, we show a few provenance semirings used in the literature [13, 17, 20] for discrete reasoning and approximated probabilistic reasoning. As an example, a tag can be a boolean formula $\phi \in \Phi$ represented in disjunctive normal form (DNF) under set notation. Here, the boolean variables v will be references to facts in the input databases. With probability $\text{Pr}(v)$ attached, one might perform top- k filtering on proofs to avoid blow-up of the boolean formulas. In order to support the discrete, probabilistic, and differentiable modes of reasoning, Lobster implements 7 commonly used provenance semirings, which we elaborate in Section 3.5.

3.2 APM: A Language for Parallel Machines

APM is a low-level, assembly-style procedural language that explicitly exposes allocations and is composed exclusively of instructions which permit massively parallel execution. An overview of the instructions is provided in Table 1. APM seeks to solve the problem that, traditionally, GPU programming is much like C programming: an unbounded set of programs can be expressed, even ones that map poorly to

(Tag)	t	$\in$	T		
(False)	0	$\in$	T		
(True)	1	$\in$	T		
(Disjunction)	$\oplus$	:	$T \times T \rightarrow T$		
(Conjunction)	$\otimes$	:	$T \times T \rightarrow T$		

(a) The provenance semiring structure.

Provenance	T	0	1	$\oplus$	$\otimes$
Bool	$\{\perp, \top\}$	$\perp$	$\top$	$\vee$	$\wedge$
Max-Min-Prob	$[0, 1]$	0	1	$\max$	$\min$
Top- k -Proofs	Φ	$\{\}$	$\{\emptyset\}$	$\vee_k$	$\wedge_k$

(b) Common examples of provenance semirings.

Figure 5. Provenance semiring structure and examples.

the underlying hardware. APM alleviates this problem by taking the implicit guidelines of the GPU programming model and making them explicit in the design of APM. This results in a desirable property: once a program is compiled to APM, efficient GPU execution is assured. We consider a number of core limitations of the GPU programming paradigm and make a corresponding design decision in APM:

1. **Lockstep Execution** While GPUs have thousands of cores available for parallel computation, these cores are not as flexible as CPU cores. Specifically, GPU cores implement a single instruction, multiple data (SIMD) paradigm, in which a set of 32 threads (known as a *warp*) must execute the same set of instructions while operating over separate thread registers. This informs the **lack of control flow** in APM, ensuring minimal thread divergence.
2. **Allocation** Allocating GPU memory while GPU code is executing has negative performance implications. Therefore, data structures commonly used in database systems that rely on pointer chasing like B-Trees and Tries are non-starters in programs wishing to execute on GPUs. Instead, data structures like sorted arrays, which use large contiguous blocks of memory and can pre-allocate enough memory for their use up-front, are preferred. This restriction is respected by requiring APM programs be in **static single assignment (SSA)** form [12], and by requiring **all registers be explicitly allocated** with a size before use. As a result, compiling to APM requires statically determining memory allocation points, and determining register lifetimes is trivial.
3. **Coalesced Memory** In GPUs, memory accesses are fastest when threads within a warp access consecutive memory locations, a pattern known as coalesced memory access. As such, a columnar representation for relational tables helps ensure maximum utilization of GPU memory bandwidth by ensuring the common path of per-column memory operations results in coalesced accesses. Correspondingly, all registers in APM are **vector registers** that store a non-resizable buffer of identically-typed values.

3.3 Compiling RAM to APM

Given the design considerations of APM, we now must determine how to compile a RAM program to APM. The most important decision is determining how to represent tables in APM, as they are the base unit of data in relational algebra. As tables consist of a fixed schema of columns of identical size, it is straightforward to represent an arity n relation as n registers of equal size, adding an additional register for provenance semiring tags. It is then sensible to discuss sets of registers in APM as tables, just as we discuss sets of facts in RAM as relations.

With table representation fixed, compiling RAM to APM involves flattening the RAM program (represented as a DAG) into the APM program (represented as a sequential list of instructions). This flattening is implemented via a recursive RAM-to-APM function $\text{compile} :: \text{RAM} \rightarrow [\text{instr}] \times [\text{reg}]$, a function that compiles a RAM expression into a sequence of instructions and returns the registers the result table is stored in. For the complete definition of compile , see the Appendix. Translation proceeds in the presence of a translation context F_T , also known as the EDB, which contains schema necessary for applying the translations and the provenance for using the proper tag operations. We now examine two of these translation rules in detail to give examples of why the translation to APM is challenging but makes the resulting programs amenable to GPU execution.

Project. Projection is an example of the simplest parallelism Lobster exploits: row-level parallelism. A projection expression $\pi_\alpha(\epsilon)$ consists of a projection expression α evaluated on an input table. Critically, the expression is evaluated identically and without coordination across each fact of the input relation: this is a perfect fit for the SIMD paradigm employed by GPUs. Propagating semiring tags is also straightforward: the provenance of each fact in the output is tied to exactly one fact in the input, so tags can be copied through to the result without modification. Finally, performing allocation of the input facts is also straightforward, as the size of the output relation is identical to that of the input relation. A concrete usage of lowering project to APM can be seen in Figure 6, which features the compilation of a simple permutation projection.

Join. Potentially the most important relational operator, $\text{join} (\bowtie_n)$ forms the computational core of most Lobster programs, so it is important to find an efficient implementation. Unfortunately, it is also more challenging than project for two reasons: (1) whereas each input fact in project produces exactly one output fact, with join each input fact can compute zero or more output facts and (2) rather than evaluating an expression against each row, join requires a membership test against the table being joined against. To overcome these obstacles, Lobster takes inspiration from GPU hash-join algorithms [38]. Lobster has an additional requirement, however,